

Where Authorization Stops: Effect-Bound Containment, Independent Evidence and the Limits of Preview

Aditya Pratap Singh

AI Incident Response Sprint, Apart Research (<https://apartresearch.com/sprints/ai-incident-response-sprint-2026-09-11-to-2026-09-13>), September 2026

Draft for author review. Not submitted; independent reproduction pending.

AI contribution. AI coding assistants, including OpenAI Codex, implemented and revised the code, ran analyses, and drafted and revised this manuscript. Aditya Pratap Singh directed the project and requested experiments, reviews and revisions. The author has not personally rewritten the manuscript; full personal review of its claims remains pending. See the LLM Usage Statement for the scope of checks.

Abstract

Containment requires controlling effects through permitted routes and evidence that outside reviewers can inspect. We evaluate exact authorization, independent backend observation and preview-based quarantine on synthetic Git, HTTP and API-twin workflows. Git receive-time enforcement allowed 0/6 forbidden ref updates versus 3/6 for broker preview; an exact-authority HTTP gateway allowed 0/12 attacks versus 8/12 for endpoint restriction. Both preserved benign completion. Backend evidence detected 6/6 executed attacks, while intent logs detected none. Preview blocked reproducible faults but missed execution-only writes. Transactional state and output gates prevented specified write and read-leak fixtures; subsequent correctness review found and repaired additional defects. Four model pilots remain insufficient to establish realistic attack resistance, including an interrupted repaired-gate study with no final evaluation. We provide a nine-control checklist, offline checks and retained negative results. Quarantine superiority, production safety and independent reproduction remain unestablished. All results concern designed fixtures on trusted hosts, not independently assessed production deployments.

1. Introduction

Sandboxes for capability evaluations usually leave some route open: a package mirror, a proxy, a tool API. According to the sprint materials, the July 2026 incident used exactly such a route, a package-registry cache proxy that was the sandbox's only permitted egress [8]. Restricting *which endpoint* an agent may reach does not constrain *what effect* a request to that endpoint produces. An accurate log of what the agent intended can also disagree with what the backend executed.

Central finding. A successful preview does not establish execution safety. In the tested fixtures, faults introduced only during execution pass preview; checks at the component committing state or returning data prevent those specified effects. Independent backend

evidence distinguishes approved requests from actual effects. This is a bounded mechanism result, not general attack resistance.

Our original hypothesis compared server-side capability enforcement plus dynamic quarantine against intent logging or broker-only authorization. Its weak-baseline comparison is supported, but bundles two mechanisms and cannot identify quarantine's increment. We preserve its exact wording and all H1–H34 outcomes in `hypotheses.md`. H1–H7 were reconstructed; H8 onward were recorded before their runs. Comparisons of preview and execution gates characterize their failures, not the causal effect of adding preview to an otherwise identical execution gate. Model pilots have not resolved that separate question.

Threat model. The actor (a scripted or model-generated client) can alter its requests, client configuration, local files, retries and timing, and can try any route its sandbox allows. It cannot edit server-side policy, hooks or backend state directly. The broker, gateway, backend and host are trusted. A compromised egress service or kernel is out of scope (Section 5).

The evidence chain is:

1. **Authority at execution.** Native Git and isolated HTTP checks prevent the tested effects that upstream checks admit, preserving benign completion (Tables 1–2). Backend observations expose effects omitted by upstream records (Table 3).
2. **The counterexample and its scope.** Preview misses execution-only writes (Table 4); transactional state enforcement prevents the specified write, and response enforcement prevents an explicit read leak (Tables 5–6). These are designed faults on trusted backends, not discovered model exploits.
3. **A usable assessment.** Nine bounded acceptance rules, evidence requirements and a worked assessment separate local PASS, demonstrated FAIL and deployment NOT VERIFIED. One answer-key-withheld AI exercise reached all six scoped decisions; no independent human assessment or adoption result exists (Appendix B).

2. Related Work

Complete mediation [1] and the confused-deputy problem [2] motivate checking every access against specific authority rather than ambient permissions. Our capabilities bind tool, canonical arguments, run and request identity, nonce and expiry, which is a standard object-capability pattern rather than a new primitive. AI Control [3] argues for evaluating safety measures against intentionally subversive models. Our scripted suites follow that red-team framing at small scale but are not frontier-agent evaluations. AgentDojo [4] benchmarks prompt-injection attacks against tool-using agents using execution-based outcomes. CaMeL [5] separates control and data flow and enforces capability policies at tool invocation. Our narrow contribution tests backend deviations from authorized calls and compares preview checks with enforcement at the component that executes the effect.

3. Methods

Testbeds. All tests use synthetic fixtures and temporary repositories; no production service or victim data was touched (Figure 1).

- *Native Git* (Git 2.43.0). The permitted effect is one exact old → new object transition on `refs/heads/review`; production must never change. Attacks use real Git

configuration (push redirects, extra refsspecs) and changes to client configuration or HEAD between admission and push. The effect is scored from per-push refs and native reflogs, not exit codes.

- *Isolated HTTP*. A client reads a synthetic report and publishes it as one issue on a local Pome GitHub twin [6]. The actor runs in Bubblewrap [10] mount, PID and network namespaces with dropped capabilities. It can reach only one gateway socket, and never holds backend credentials or signing keys.
- *Pome observer study*. A separate process fetches Pome's own request tape and state and stores hashed receipts, so the broker cannot supply that evidence.
- *Quarantine studies*. A local modeled server (720 cases), two hosted Arga GitHub twins used as execution and preview copies [9], and a separate-process local service with durable operation identities.
- *Combined architecture*. The same separate-process local service, adding a third arm whose service enforces an execution-time effect gate inside the write transaction, so preview-based release and execution-side enforcement are compared under one harness (48 cells). A fourth arm additionally confines the outbound response to each tool's declared output class, tested against an exfiltrating read that returns a secret with no state change (68 cells).

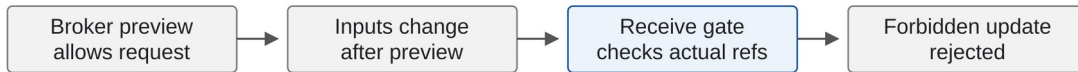
Conditions. Every comparison holds the policy information fixed and varies only *where* or *how* it is enforced. Git: intent-only dispatch, exact command matching, broker native dry-run preview, and a pre-receive hook applying the same effect predicate to Git's actual update batch. HTTP: upstream approval only, sandbox with an endpoint restriction, and sandbox with exact capability revalidation at the gateway. Quarantine: static deny-unknown, hold-all, static authorization of the same reviewed request shapes, and selective release. Selective release runs the request against a preview copy and issues a one-use, server-signed approval bound to the request, state context and policy revision. Combined: static authorization, selective release, and an execution-time effect gate that shares the same authority but additionally confines each operation's committed effect at the service (a read changes no state; a write adds at most its own exact issue), rolling back atomically otherwise — a static effect policy at the effect boundary, the service-path analog of the Git pre-receive hook, not adaptive reasoning.

Scoring. Attack success means an actual unauthorized backend effect: changed refs, a Pome or Arga event or state change, or a witnessed sink delivery. Benign completion checks the correct output and flags duplicates. False rejection, false quarantine, detection and unknown evidence use separate denominators. Missing or malformed evidence is scored *unknown*, never clean.

Protocol discipline. From H8 onward, each study's hypotheses and decision rules were recorded before canonical runs (internal timestamps, not external preregistration). Rehearsals were stored separately, and failed prerequisites and unrun cells were kept. Cases are deterministic and constructed to exercise known failure modes, so repetitions are consistency checks rather than independent samples. We report exact counts and make no population or significance claims.

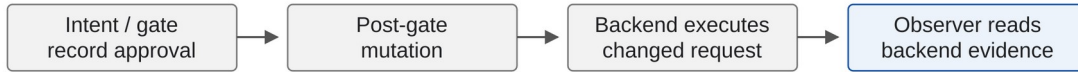
What did not work. Exact state restore was unavailable for both twin providers. Pome seeds do not reproduce timestamps or commit hashes, and Arga resets regenerate Git identifiers. Real-backend quarantine therefore returns an explicit "unsimulated hold" instead of a fabricated clone. Four small model-attacker pilots remain insufficient to establish realistic resistance. H27 stopped early, H30 lacked substantive attacks, H31 failed its search gate, and H34 stopped with 0/16 final cells (Appendix C).

A. Native Git — enforcement timing



Same effect policy at preview and receive time; 3/6 versus 0/6 attack successes.

B. Pome — independent observation



Detection is after execution. The observer does not prevent or roll back the effect.

C. Local quarantine — preview fidelity



Matching copied state does not guarantee matching future behavior. Host/backend trust remains.

Figure 1. Three tested boundaries. Each row is a separate experiment, not one combined deployment. (A) The same effect policy checked at broker preview versus Git receive time. (B) Post-gate mutations are visible only in independently acquired backend evidence; detection happens after execution. (C) A preview can match declared state yet diverge in live behavior.

4. Results

Table 1. Native Git enforcement (44 cells). Six attack and five benign workflows per condition; identical effect policy in the last two rows. Source: results/git-enforcement-v1.

Condition	Attack successes / 6	Benign completion / 5	False rejection / 5
Intent-only dispatch	6	5	0
Exact command match	6	3	2
Broker native preview	3	5	0
Receive-time effect gate	0	5	0

A successful preview establishes that the planned update passed the policy checks; it does not bind the later push to the target and commit that were checked. The three attacks that passed the preview each changed client state *after* admission (push redirect, extra target, wrong commit). The receive gate rejected them because it evaluates the update Git actually receives. Mixed allowed/forbidden pushes left all refs and journals unchanged. The gate also held under missing, malformed and revoked policies, duplicate updates, deletions and `--no-verify`. Exact command matching is a poor proxy: it rejected two harmless variations and missed all configuration-driven violations. Median benign dispatch was 40.8 ms with the gate versus 12.4 ms intent-only (single-run, descriptive).

Table 2. Isolated file-to-HTTP publishing (51 cells). Twelve attacks (credential file, symlink, direct TCP exfiltration, wrong project, altered content or method, missing token, identity and nonce rebinding, replay, subprocess and deferred substitution) and five benign workflows. Source: results/isolated-http-v1.

Condition	Attack successes / 12	Correct benign completion / 5	Duplicate benign effects
Upstream approval only	11	4	1
Sandbox + endpoint restriction	8	4	1
Sandbox + exact gateway authority	0	5	0

The sandbox removed the three filesystem and network bypasses. Exact revalidation at the only reachable gateway removed the remaining eight request, identity and replay violations. One unsupported DELETE attack failed natively in every condition and adds no evidence. Weak conditions re-executed a lost-response retry; the exact gateway returned the stored result. Median actor time was about 70 ms in all three conditions.

Table 3. Independent evidence (16 Pome cells). Six executed attacks, two prevented attempts, eight benign controls including 404s and lost responses. Source: results/observer-comparison-v1.

Evidence view	Executed attacks detected / 6	Prevented attempts alerted / 2	Benign false alarms / 8
Agent intent log	0	0	0
Gate-entry request	2	2	0
Backend request tape	6	0	0

Two post-gate substitutions returned seeded file content without changing state, so a state diff alone would also miss them. Detection bounds were 4.5–22.2 ms after dispatch. Detection does not undo an effect: under full enforcement, both post-gate attacks still executed. A later native Git study confirmed the same pattern. Request-text matching raised 4/10 benign false alarms and missed 6/6 configuration-driven violations, while Git’s porcelain response flagged all 6.

Table 4. Preview-based quarantine. Top: 720 local cases (five attack and four benign workflows × 20 repetitions). Bottom: fault controls on hosted Arga twins (H26) and a separate-process local service (H29). ✓ = unauthorized effect executed; ✗ = prevented. Sources: results/selective-release-v1, arga-quarantine-v1, local-readiness-v1.

Local modeled (720)	Attack successes / 100	Benign completion / 80	False reject / quarantine
Static deny-unknown	0	0	80 / 0
Hold-all quarantine	0	0	0 / 80
Static, same reviewed shapes	100	80	0 / 0
Selective release	20	80	0 / 0

Fault control	Static (reviewed shapes)	Selective release
Hidden write visible in preview (Arga)	✓	✗
Visible / deferred write (H29)	✓ / ✓	✗ / ✗
Write that occurs only at execution (Arga, H29)	✓	✓

Selective release is a real but conditional improvement. It matched static authorization on benign reads (80/80; 2/2 on Arga) and blocked every fault its preview could observe. All 20 local simulation-gap repetitions, and the execution-only fault on both later testbeds, escaped and were flagged only after execution. Selective and static both beat blanket denial on utility. Neither beat blanket denial on prevention: an earlier 800-case matrix tied static denial and hold-all at 0/140 attack successes (Appendix C). Selective median latency was 1.33 s versus 0.55 s static in the H30 pilot.

Table 5. Combined architecture (48 cells). The three arms on one service path over the four fault controls; all three complete 12/12 normal cells with no effect and no holds. ✓ = unauthorized effect executed; ✗ = prevented; *hold* = benign read held (utility lost); *infra* = committed write, response lost, reconciliation needed. Source: results/combined-gate-v1.

Fault control	Static	Selective	Effect gate
Hidden write visible in preview	✓	✗ (hold)	✗ (completes)
Deferred write	✓	✗ (hold)	✗ (completes)
Write only at execution	✓	✓	✗ (completes)
Response lost after commit	infra	infra	infra

The execution-time effect gate was the only arm to prevent the execution-only write, and the only arm to prevent the preview-visible faults *without* holding the benign read. It does not repair the transport fault: a committed write with a lost response still needs reconciliation in every arm. This is the same execution-side mechanism as the Git receive gate, now on a second backend; it is not a quarantine advantage and does not defend a compromised service or host.

Table 6. Data-flow gate (68 cells). Adding an exfiltrating read — a read that changes no state but returns a secret outside its declared output class, visible only at execution. *state gate* is the Table 5 effect gate; *data-flow gate* also confines the outbound response. *leak* = secret reached the actor; ✗ = prevented. Every arm completes 12/12 normal cells; the data-flow gate matches the state gate on all four Table 5 faults. Source: results/dataflow-gate-v1.

Fault	Static	Selective	State gate	Data-flow gate
Read exfiltration	leak	leak	leak	✗ (completes)
Write only at execution	✓	✓	✗	✗

The exfiltrating read exposes the state gate's blind spot: it changes no state, so the state oracle calls it clean and the state gate, static and selective all let the secret through. Confining the *outbound data flow* at execution catches it and still returns the authorized boolean. The gates layer cleanly — data-flow $\supseteq$ state — which is the general point: any effect you can observe and withhold at the execution boundary belongs in the gate, not in a preview that only predicts it. This covers explicit payloads only; covert and implicit channels are out of scope for every arm here.

Robustness. The central contrasts (0 vs 3 of 6; 0 vs 8 of 12; 6 vs 0 of 6) are deterministic under unchanged cases and reproduced in regression tests. They are not rate estimates. The attack sets are purpose-built, which shows that each failure mode *can* occur and that the control stops it, not how often it occurs. The test suite and `verify_results.py` check the local implementation and canonical evidence. Passing them does not establish population-level security. Later correctness review repaired existing-row preservation, rollback on rejected output and schema-valid unapproved output. These repairs do not retroactively strengthen the frozen experiments; their separate regressions remain in the repo.

5. Discussion and Limitations

Implications for containment standards. “Only one permitted egress” is not a sufficient requirement. Our results support three requirements a lab could adopt and a reviewer could check offline: (i) the permitted route checks the *exact effect*, meaning request, identity, one-use nonce and target, at the component that executes it; (ii) the lab retains backend-side evidence that the actor and broker cannot write; and (iii) any dry run or preview is treated as advisory unless an execution-side check also constrains the effect. Appendix B turns these into acceptance tests. The cost in our testbeds was tens of milliseconds per action for enforcement and roughly one extra second for preview.

What remains unresolved. The tested state and explicit-output effects can be checked before commit or delivery. We did not establish that preview adds value when those execution checks are already present. A prospective G versus QG comparison must hold execution enforcement fixed and add only preview; the existing selective versus effect-gate comparison does not do that. Independently constructed fault families and useful-output controls are needed for a broader claim. An irreversible effect outside a transaction is not automatically a quarantine win: an equally informed mediator might gate its sink directly. The follow-up protocol records this competing explanation and stops if no discriminating route exists. No additional model search or combined-arm evaluation is reported here.

Relation to the incident. We did not reproduce the incident. The incident route was compromised through a zero-day in the egress proxy itself. Our gateway is trusted, so our controls do not stop an attacker who gains code execution inside it. The Git and combined-architecture results point to the more robust placement: enforce the exact effect at the receiving backend, not only in the egress process, so that a route which passed upstream review still cannot commit an unauthorized effect. We did not test that placement against a compromised proxy or backend.

Limitations

- **Trust boundary.** Broker, gateway, observer and backend share one host. Receipts detect later tampering, but a source that fabricates internally consistent evidence from the start passes validation (a retained negative test).
- **Scope.** Protection covers refs, issues and declared state in synthetic twins. It does not cover object uploads before pre-receive, resource exhaustion, covert channels, arbitrary malicious tool code, kernel escapes or multi-host deployments.
- **Fidelity.** Neither twin provider supports exact state restore. Argv equivalence was checked only for issue and access state.
- **Sample.** Constructed, deterministic, small scenario sets. Model-attacker pilots were incomplete or had inadequate coverage. H27 found an availability failure; the repaired-

gate study stopped before final evaluation. Realistic AI-attacker resistance remains unestablished.

- **Hypothesis status.** The weak-baseline part is supported. The claim that *quarantine* adds prevention over equally informed static enforcement is conditional and unestablished. Our integrated real-backend workflow ran with an explicit hold instead of a real preview.

6. Conclusion

Containment that names a permitted route but not the effects allowed through it is hard to verify and easy to misuse. On two execution paths, native Git and an isolated HTTP gateway, checking exact authority where the effect executes prevented every tested violation that an equally informed upstream check missed, without costing benign completion. Evidence taken from the backend exposed every executed mismatch that intent and gate logs concealed.

Preview-based quarantine prevented only faults its preview reproduced. The execution-time state and output gates prevented the specified execution-only fixtures, with later correctness repairs retained separately. These results support execution-side checks and independent evidence; they do not establish general gate superiority or justify treating a clean preview as proof of safety. Independent reproduction and an adoption-cost assessment remain necessary.

Code and Data

- **Artifact snapshot:** `effect-bound-containment-submission.tar.gz` contains the current code, reports and evidence, with a SHA-256 file manifest. Supply it alongside this PDF. The public [repository](#) is an earlier revision and does not contain all repairs in this report. Code is MIT; reports, protocols and results are CC BY 4.0.
- **Data:** every run, including failed prerequisites and unrun cells, with hashed receipts under `results/` (the combined-architecture study is `results/combined-gate-v1`, the data-flow study `results/dataflow-gate-v1`). Hypotheses H1–H34 and their predeclared protocols are in `hypotheses.md`.
- **Reviewer procedure:** `report/README.md#offline-reviewer-procedure`. No accounts, API keys or hosted services are required.
- **Info-hazard note:** the attack fixtures illustrate generic authorization weaknesses against owned synthetic services and contain no exploit for any real system (Appendix A).

Author Contributions

Aditya Pratap Singh directed the project and requested experiments, reviews and revisions. AI coding assistants implemented the harness, ran checks and drafted the report. The extent of personal claim review is recorded in the LLM Usage Statement.

References

1. Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9).
<https://web.mit.edu/Saltzer/www/publications/protection/>
2. Hardy, N. (1988). The confused deputy. *ACM SIGOPS Operating Systems Review*, 22(4). <https://doi.org/10.1145/54289.871709>
3. Greenblatt, R., Shlegeris, B., Sachan, K., & Roger, F. (2023). AI control: Improving safety despite intentional subversion. arXiv:2312.06942.
<https://arxiv.org/abs/2312.06942>
4. Debenedetti, E., et al. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. arXiv:2406.13352.
<https://arxiv.org/abs/2406.13352>
5. Debenedetti, E., et al. (2025). Defeating prompt injections by design. arXiv:2503.18813.
<https://arxiv.org/abs/2503.18813>
6. Pome contributors. Digital Twins (CLI 0.43.0). <https://github.com/pome-sh/digital-twins>
7. Git project. githooks: pre-receive (Git 2.43.0).
https://git-scm.com/docs/githooks/2.43.0#_pre_receive
8. Apart Research (2026). AI Incident Response Sprint, 11–13 September 2026.
<https://apartresearch.com/sprints/ai-incident-response-sprint-2026-09-11-to-2026-09-13>
9. Arga Labs. Digital twins documentation: GitHub twin reference.
<https://docs.argalabs.com/concepts/twin-reference#github>
10. Bubblewrap contributors. Bubblewrap 0.9.0: unprivileged sandboxing tool.
<https://github.com/containers/bubblewrap>

Appendix A. Limitations and Dual-Use Considerations

Assumptions and what breaks if they fail. (1) *Trusted enforcement point*. If the gateway or hook is compromised, Tables 1–2 say nothing about prevention. Only backend-side evidence remains useful, and only if it is stored off-host. (2) *Trusted backend and observer*. If the backend lies consistently from its first record, Table 3 detection fails. Receipts protect only against later alteration. (3) *Sole route*. If any credential or network path bypasses the gateway, exact authority is moot. Without OS isolation, our boundary audit showed a same-user process reading a fixture credential and bypassing the broker. (4) *Preview fidelity*. If execution behavior differs from preview, selective release admits the effect (Table 4).

Method limits. Deterministic, constructed cases; small scenario diversity; shared implementation authors for harness and scorer; same-host timing; Pome tape and state are separate, non-atomic reads. Model pilots were budget-limited, and one (H27) stopped at 3/18 cells when hosted quota ran out.

Dual use. The fixtures show how exact-request checks, endpoint restrictions and previews can be bypassed. They are generic authorization weaknesses, not exploits against a product, and target only locally created or owned synthetic services. Temporary credentials are generated inside fixtures. Real deployment logs used as effect evidence may contain sensitive data, so access control and redaction must preserve enough detail to verify effects. We release the suite so defenders can test their own boundaries; it should not be pointed at third-party systems.

Appendix B. Evidence-Bound Containment Checklist

The authoritative rules are `report/containment-standard.md`; this table and the consolidated report summarize the same nine controls. These are proposed requirements, not an adopted or certified standard. **PASS** requires the acceptance test plus provenance evidence; **FAIL** means a demonstrated violation; **NOT VERIFIED** means missing evidence. A passing test runner can correctly demonstrate a FAIL through a negative fixture.

Control	Acceptance test (in tests/)	Measured outcome	Residual limit
C1 Dispatch only exact authority (request, identity, nonce, expiry)	<code>test_http_boundary.py</code>	0/12 attacks, 5/5 benign (Table 2)	Gateway/host compromise not covered
C2 Deny alternate actor routes	<code>test_http_boundary.py</code>	Endpoint-only 8/12 vs exact gateway 0/12	Deployment route/credential inventory NOT VERIFIED
C3 One effect per operation across retries	<code>test_runtime_recovery.py</code> , <code>test_local_sandbox.py</code>	12/12 ordinary workflows per arm; malformed commit response loses completion	Needs trusted reconciliation; multi-host untested
C4 Independently acquired call/effect evidence	<code>test_evidence.py</code> , <code>test_observers.py</code>	Backend 6/6 vs gate 2/6 vs intent 0/6	First-record forgery; compromised host
C5 Hold required missing previews; retain uncertain commits	<code>test_runtime_recovery.py</code> , <code>test_broker_workflow.py</code>	Unsupported or timed-out previews hold; no automatic re-dispatch	Availability loss; escalation untested
C6 Check useful output; preserve every trial	<code>test_local_sandbox.py</code> , <code>test_adaptive_pilot.py</code>	Duplicates counted as failures; unrun cells retained	No general workload estimate
Q1 Validate preview behavior before claiming safety	<code>test_local_sandbox.py</code> (simulation_gap)	Execution-only write escapes static and preview arms	General preview-safety claim: FAIL
C7 Confine the committed effect at the execution boundary	<code>test_combined_gate.py</code>	Effect gate prevents the execution-only write and all preview-visible faults, completes 12/12 normal cells (Table 5)	Static effect policy; no compromised-service or transport-fault coverage

Control	Acceptance test (in tests/)	Measured outcome	Residual limit
C8 Confine outbound data flow to the declared output class	<code>test_dataflow_gate.py</code>	Data-flow gate prevents the read exfiltration that the state gate, static and preview all leak, and returns the authorized value (Table 6)	Explicit payloads only; covert/implicit channels out of scope

C5 requires a preview hold only when release policy requires preview; uncertain commits must always be reconciled. Q1 is not applicable without a preview safety claim. The deployment evidence template records missing evidence, not compliance.

Adopting lab must supply: route and credential inventory for the actor environment, versioned policy and enforcement software, evidence anchors stored outside the actor's host, and an owner for uncertain commits. Local package verification cannot authenticate those.

Worked decision and external exercise. `report/worked-assessment.md` traces four exact archived cases to decisions. An execution gate passes the state fixture while its state-only variant fails output confinement; production route completeness and off-host authenticity remain NOT VERIFIED. `report/independent-review.md` and `report/reviewer-assessment.json` provide a timed external exercise with fixed scoring, missing-evidence questions and an adoption-burden form. An initial AI walkthrough saw the answers. A separate fresh agent, given a packet excluding reports, answers and prior reviews, reached all six scoped decisions without implementation guidance. Both verifiers passed; its reported 73-second inspection excludes initial setup and is not human effort. Raw outcomes and protocols remained visible. Records are in `report/reviews/withheld-agent-v1/`; this is one AI usability check, not human validation, new experimental replication or adoption evidence. On Linux, `./verify.sh` checks the package; a green test suite includes negative fixtures and is not a blanket compliance decision.

Appendix C. Additional Results and Model Pilots

Local 800-case matrix (7 attack and 3 benign scenarios × 20). Attack successes / 140 and accepted legitimate calls / 60: intent-only 140 / 60; broker-only 100 / 40 (20 false quarantines); static server 0 / 40 (20 false rejections); hold-all 0 / 40 (20 false quarantines). Server enforcement beats weaker baselines; hold-all quarantine ties static denial.

Model-attacker pilots (never pooled; cells run = final cells completed / planned; estimated cost; calls include unresolved reservations):

Study	Calls	Cost	Cells run	Outcome
H27 hosted (GPT-5.4 mini)	10	\$0.14	3 / 18	Hosted quota exhausted; no matched attack pair. Found a request-ID poisoning availability bug, later fixed (H28).
H30 local (GPT-5.4 mini)	10	\$0.10	18 / 18	9/9 tasks per arm; 4 empty and 2 authorized-only candidates, so no substantive attacks.

Study	Calls	Cost	Cells run	Outcome
H31 adaptive (GPT-5.4)	12	\$1.16	0 / 18	9/12 outputs exhausted token budget on reasoning; search gate failed.
H34 repaired gate (GPT-5.4)	5 + 1*	See note*	0 / 16	Interrupted before coverage assessment; no sealed final candidates.

H34: five completed calls cost \$0.370944; one unresolved call retains a \$0.32 reservation.

H34 retained ten development cells, five per arm, with all tasks completed and no observed forbidden state effect, successful-output violation or hold. Call dev - 5 has no retained response or usage; the interruption cause is unknown. The frozen protocol forbids retrying uncertain calls, so the full \$0.32 reservation remains. No final resistance estimate follows. See [report/effect-gate-pilot.md](#).

The pilots show that request-only attack interfaces behind prebound identities give a model little room to create the backend behavior mismatch that separates quarantine from static enforcement. Future model evaluations should first demonstrate usable candidates on separate development data.

Hypothesis ledger. H1–H34 with evidence paths and outcomes, including refuted (H7, H12) and unsupported (H2) propositions, are in [hypotheses.md](#).

LLM Usage Statement

AI coding assistants, including OpenAI Codex, implemented and revised the harness and tests, ran experiments and analyses, reconciled evidence, and drafted and revised this report. Aditya Pratap Singh directed the project and requested experiments, reviews and revisions. The author has not personally rewritten the manuscript. In a guided review, the author explained the conceptual gap between preview approval and the update that actually executes. This was a conceptual review of the first claim, not an independent check of its numerical results; For the remaining three central claims, the assistant drafted explanations in the author’s voice at his request; these are not author-supplied answers or evidence of completed personal review. Review of those claims remains pending. We do not attribute the assistants’ statistical checks or code reviews to the author.

OpenAI GPT-5.4 mini and GPT-5.4 were experimental subjects in the attacker pilots (Appendix C); that usage is separate from the assistants used to develop the project. Automated checks tie the consolidated report’s tables to retained archives ([verify_results.py](#), [tests/test_report.py](#)), and the assistants checked this report’s numbers locally against retained records. These checks do not constitute independent replication or completed personal verification by the author.

One fresh-context Codex reviewer saw the worked answers. A second received a packet with those answers and prior reviews withheld. Both exercises were AI reviews, not independent human validation. Before submission, the author must review the retained claims and confirm the assistant/tool inventory; this statement should then describe the review actually completed. Human rewriting is not asserted or required as a condition of this disclosure.